

Parameterisierte Algorithmen

WiSe 2025/26 in Trier

Henning Fernau

fernau@uni-trier.de

Übungsbetreuung: Kevin Mann, mann@uni-trier.de

Parameterisierte Algorithmen

Gesamtübersicht

- Einführung
- Grundbegriffe
- Problemkerne
- Suchbäume
- Graphparameter
- Weitere Methoden
- Komplexitätstheorie—parameterisiert

Suchbäume: Was heute auf dem Plan steht

- Planare Graphen als Beispiel einer speziellen Graphklasse
 - Unabhängige Mengen
 - Dominierende Mengen
 - Facettenüberdeckungen
- Nochmals: Suchbaumstrategien an einem Clustering-Problem
- Measure & Conquer: Verbesserte Laufzeitanalysen für Suchbäume

Bisher ... heute

Suchbäume erlauben (oft recht) effiziente parameterisierte Algorithmen.

Graphprobleme können einfacher sein, wenn sie auf gewisse Graphklassen eingeschränkt sind. Hierzu man auch leicht eine eigene Vorlesung anbieten.

`https://graphclasses.org/`

zeigt Beispiele, einschließlich der Frage, ob klassische Probleme wie INDEPENDENT SET auf der jeweiligen Graphklasse in Polynomzeit lösbar ist.

Das letzte Mal haben wir dazu Kantengraphen betrachtet.

Mehr hierzu unter: `https://graphclasses.org/classes/gc\_249.html`

Heute werden wir uns **planare Graphen** ansehen.

Vgl. auch: `https://graphclasses.org/classes/gc\_43.html`

Eulers Formel: PIS

PLANARE UNABHÄNGIGE MENGE (PIS)

Eingabe: ein planarer Graph $G = (V, E)$

Parameter: eine natürliche Zahl k

Frage: Gibt es eine unabhängige Menge $I \subseteq V$ mit $|I| = k$?

Hinweis: IS W[1]-schwer im Allgemeinen, polynomiell auf Kantengraphen.

Außerdem ist PIS NP-schwer. ABER es gilt FPT-Mitgliedschaft:

Satz 1 *Durch triviales Verzweigen bei kleingradigen Knoten: $\mathcal{O}^*(6^k)$.*

Algorithm 1 A simple search tree algorithm for PIS, called PIS-ST-simple

Input(s): planar graph $G = (V, E)$, positive integer k

Output(s): YES if G has an independent set I with $|I| = k$; NO otherwise

if $k \leq 0$ **then**

 return YES

else if $V = \emptyset$ **then**

 return NO

5: **else**

 Let v be a vertex of lowest degree in G .

 {One vertex from $N[v]$ will be in any maximal independent set.}

for all $x \in N[v]$ **do**

if PIS-ST-simple($G[V \setminus N[x]], k - 1$) **then**

10: return YES

 {All trials have failed.}

 return NO

Mehr Mathematik: PIS

Satz 2 (Aksionov, Borodin, Mel'nikov, Sabidussi, Stiebitz, Toft: Deeply asymmetric planar graphs, *J. Combinatorial Theory, Series B*, Band 95, Seiten 68–78, 2005) *Jeder zusammenhängende planar eingebettete Graph mit wenigstens zwei Knoten besitzt*

1. *entweder zwei Knoten mit Gradsumme höchstens fünf*
2. *oder zwei Knoten vom Abstand höchstens zwei mit Gradsumme höchstens sieben*
3. *oder ein Dreieck mit zwei inzidenten Knoten mit Gradsumme höchstens neun*
4. *oder zwei über eine Kante $\{u, v\}$ benachbarte Dreiecke (Dreiecksfacetten) mit Gradsumme von u und v höchstens elf.*

Hinweis Damit verbesserte Laufzeit: $\mathcal{O}^*(5.17^k)$.

Kleiner Ausflug in die Graphentheorie

Ein Graph heißt *asymmetrisch*, wenn er einen nicht-trivialen Automorphismus besitzt.

Triviales Beispiel: Ein Graph mit zwei isolierten Knoten ist asymmetrisch.

Erdős und Rényi haben 1963 gezeigt: Fast alle Graphen sind asymmetrisch. Überdies gilt: Ist $s(n)$ die Maximalzahl an Kanten, die zu einem vorgelegten Graphen mit n Knoten hinzugefügt oder gelöscht werden müssen, um einen asymmetrischen Graphen zu erzeugen, so gilt:

$$s(n) \leq \left\lfloor \frac{n-1}{2} \right\rfloor \quad \text{sowie} \quad \lim_{n \rightarrow \infty} \frac{s(n)}{n} = \frac{1}{2}$$

Insofern ist das erwähnte Ergebnis von Aksionov et al. wichtig, zeigt es doch:

Satz 3 *Jeder planare Graph $G = (V, E)$ mit wenigstens zwei Knoten enthält eine Kantenmenge A , $|A| \leq 5$, sodass $H = (V, E \setminus A)$ zwei Knoten enthält, deren Transposition einen Automorphismus auf H induziert.*

Algorithm 2 A more advanced search tree algorithm for PIS, called PIS-ST

Input(s): planar graph $G = (V, E)$, positive integer k

Output(s): YES if G has an independent set I with $|I| = k$; NO otherwise

if $k \leq 0$ **then**

 return YES

else if $V = \emptyset$ **then**

 return NO

5: **else**

 Let v be a vertex of lowest degree in G .

 {One vertex from $N[v]$ will be in any maximal independent set.}

if $\deg(v) \leq 4$ **then**

for all $x \in N[v]$ **do**

10: **if** PIS-ST($G[V \setminus N[x]], k - 1$) **then**

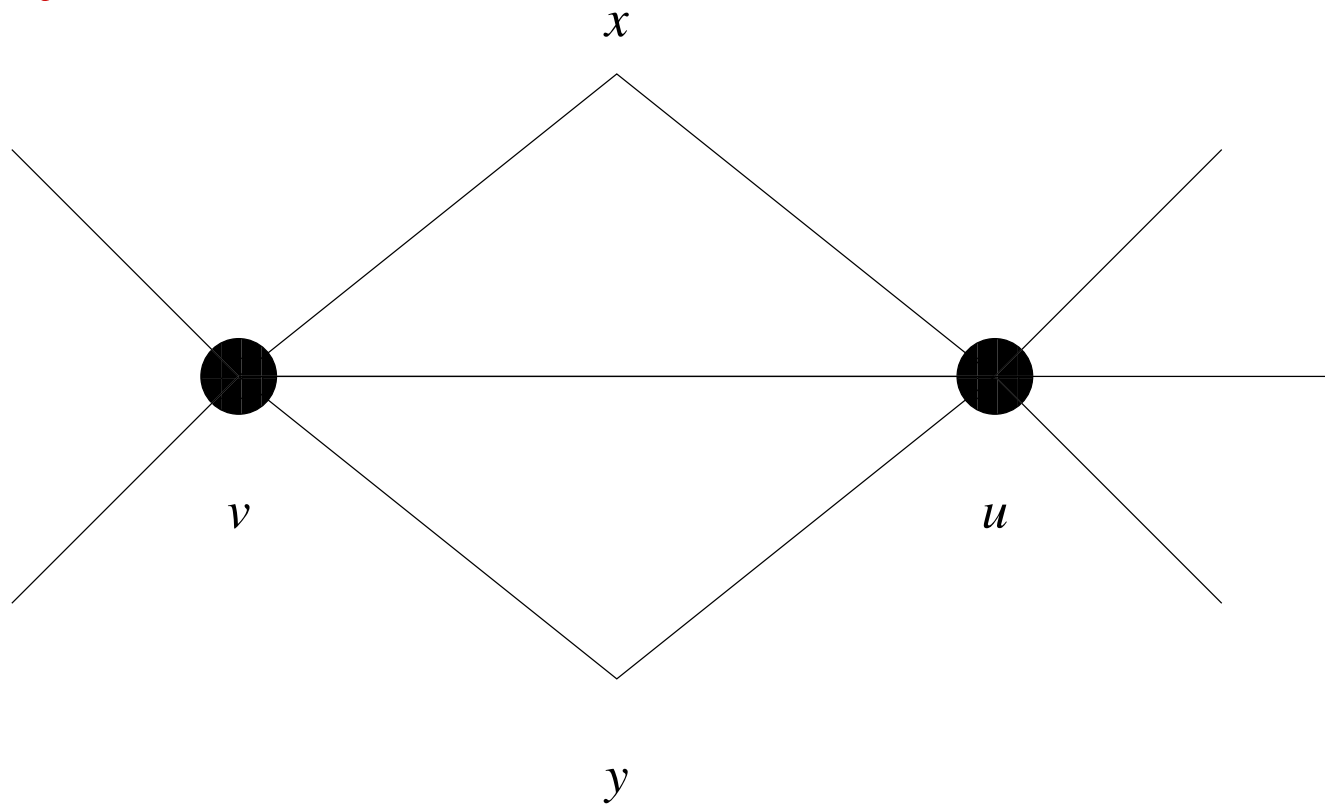
 return YES

 return NO

else

 {The fourth case of Theorem 2 applies. Details to follow. . . }

Der vierte Fall



```

{ $N(v) = \{x, y, u, v_1, v_2\}$  with  $N(u) = \{x, y, v, u_1, u_2, u_3\}$ }
if PIS-ST( $G[V \setminus N[v]]$ ,  $k - 1$ ) then
    return YES; {Branch 1:  $v$  in IS ?}
else if PIS-ST( $G[V \setminus N[u]]$ ,  $k - 1$ ) then
5:   return YES; {Branch 2:  $v$  is not in IS, but  $u$  ?}
else if PIS-ST( $G[V \setminus N[x]]$ ,  $k - 1$ ) then
    return YES; {Branch 3:  $v, u$  is not in IS, but  $x$  ?}
else if PIS-ST( $G[V \setminus N[y]]$ ,  $k - 1$ ) then
    return YES; {Branch 4:  $v, u, x$  is not in IS, but  $y$  ?}
10: else
    {Now  $v, u, x, y$  is not in IS, so one of the  $v_i$  and one of the  $u_j$  must be in a
    maximal independent set.}
    for all  $i = 1, 2; j = 1, 2, 3$  do
        if PIS-ST( $G[V \setminus (N[v_i] \cup N[u_j])]$ ,  $k - 2$ ) then
            return YES
15:   return NO

```

Zur Algorithmenanalyse

Korrektheit: Folgt mit dem Satz von Aksionov et al.

Laufzeitabschätzung: T erfüllt

$$T(k) \leq 4T(k-1) + 6T(k-2).$$

Das liefert die Behauptung:

$$c^k \approx 4c^{k-1} + 6^{k-2}$$

$$c^2 - 4c + 6 = 0$$

$$\leadsto c = 2 + \sqrt{10} \leq 5.17.$$

Waren das alle Fälle??

Gefahr: Suggestive Bilder: Nicht alle Knoten v_1, v_2 und u_1, u_2, u_3 müssen verschieden sein!

Gilt beispielsweise $v_1 = u_1$, so wäre unsere Verzweigung schlechter abzuschätzen durch:

$$T(k) \leq 5T(k-1) + 2T(k-2) < 5.4^k$$

Hier hilfreich: Separatoranalyse, denn das neue Dreieck $u, v, u_1 = v_1$ muss nicht-leeren Teil des Graphen enthalten.

Wenn u, v oder $u_1 = v_1$ in die unabhängige Menge kommen, so kann man daher Graphteile unabhängig voneinander und nacheinander durchrechnen und schon im Voraus die Parameter entsprechend erniedrigen. Eine einfache Analyse führt so auf die Rekursion:

$$T(k) \leq 2T(k-1) + 5T(k-2) < 4^k$$

Der Fall $N(v) \subset N[v]$ ist noch besser!

Aufgabe: Führen Sie eine genaue Laufzeit-Analyse der einzelnen (fehlenden) Fälle durch.

Ein komplizierteres Beispiel: PDS

PLANARE DOMINIERENDE MENGE (PDS)

Eingabe: ein planarer Graph $G = (V, E)$

Parameter: eine natürliche Zahl k

Frage: Gibt es eine dominierende Menge $D \subseteq V$ mit $|D| \leq k$?

Hinweis: Problemkern von $67k$ (sehr kompliziert)*

Problem beim Verzweigen: es entstehen zwei Arten von Knoten:

- Solche, die noch abgedeckt werden müssen und
- solche, die man schon dominiert hat, die aber noch in die Dominierungsmenge aufgenommen werden sollten.

*Siehe Chen, F., Kanj, Jia, SIAM J. Computing, Band 37, Seiten 1077–1108, 2007.

Eine Verallgemeinerung durch Annotation

PLANARE ANNOTIERTE DOMINIERENDE MENGE (PADS)

Eingabe: ein planarer Graph $G = (V, E)$ und Knotenmenge B

Parameter: eine natürliche Zahl k

Frage: Gibt es eine Menge $D \subseteq V$, $|D| \leq k$, die jeden Knoten aus B dominiert ?

Wir nennen die Knoten aus B schwarz (black) und deren Komplement W (weiß).

M.a.W.: Wir betrachten planare Schwarz-Weiß-Graphen.

Frage: Gibt es stets kleingradigen schwarzen Knoten ?

„Ohne Weiteres“ nicht: Betrachte Sterngraph, bei dem nur das Zentrum schwarz.

Problemlösung Reduktionsregeln

R1: Lösche Kanten zwischen weißen Knoten.

R2: Lösche weiße Knoten vom Grad 1.

R3: Sei u ein weißer Knoten vom Grad 2 mit schwarzen Nachbarn v_1 und v_2 .

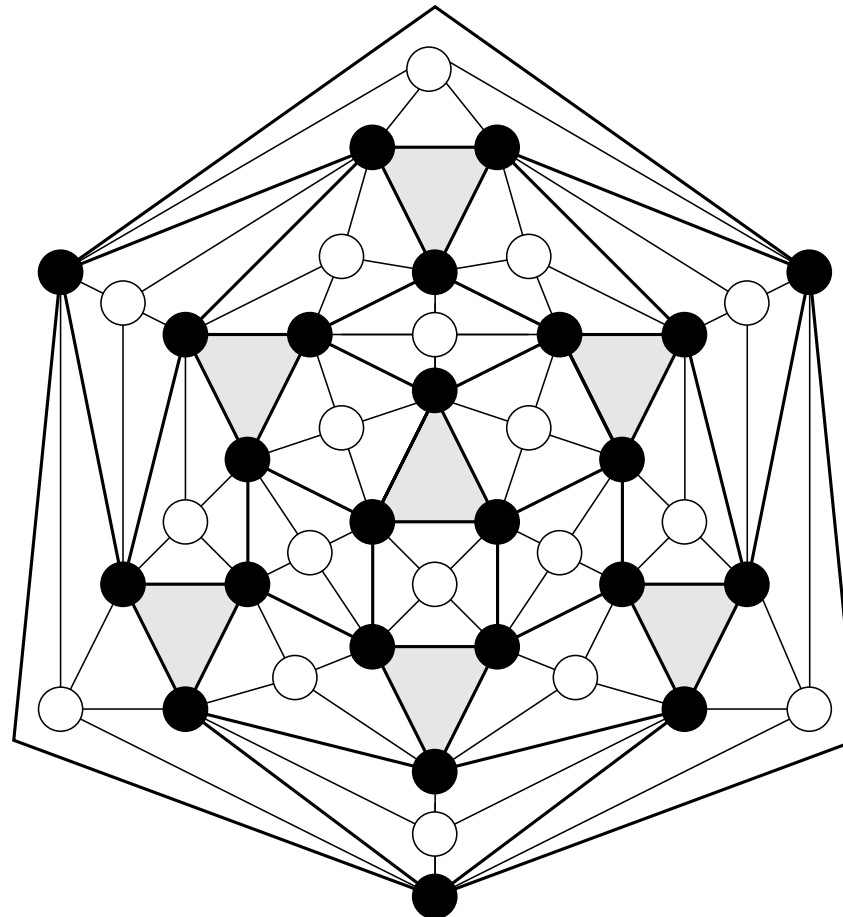
R3(a): Gibt es Kante zwischen v_1 und v_2 , so lösche u .

R3(b): Gibt es einen weiteren Knoten $v_3 \in N(v_1) \cap N(v_2)$, so lösche u .

R4: Sei u ein weißer Knoten vom Grad 3 mit schwarzen Nachbarn v_1, v_2, v_3 .
Gibt es Kanten $\{v_1, v_2\}$ und $\{v_2, v_3\}$, so lösche u .

Lemma 4 *Die Reduktionsregeln sind korrekt.*

Satz 5 *In einem reduzierten planaren Schwarz-Weiß-Graphen gibt es stets einen schwarzen Knoten vom Maximalgrad sieben.*



Ein irreduzibles Beispiel

Ein „echtes“ planares Problem

FACETTENÜBERDECKUNG (FACE COVER FC)

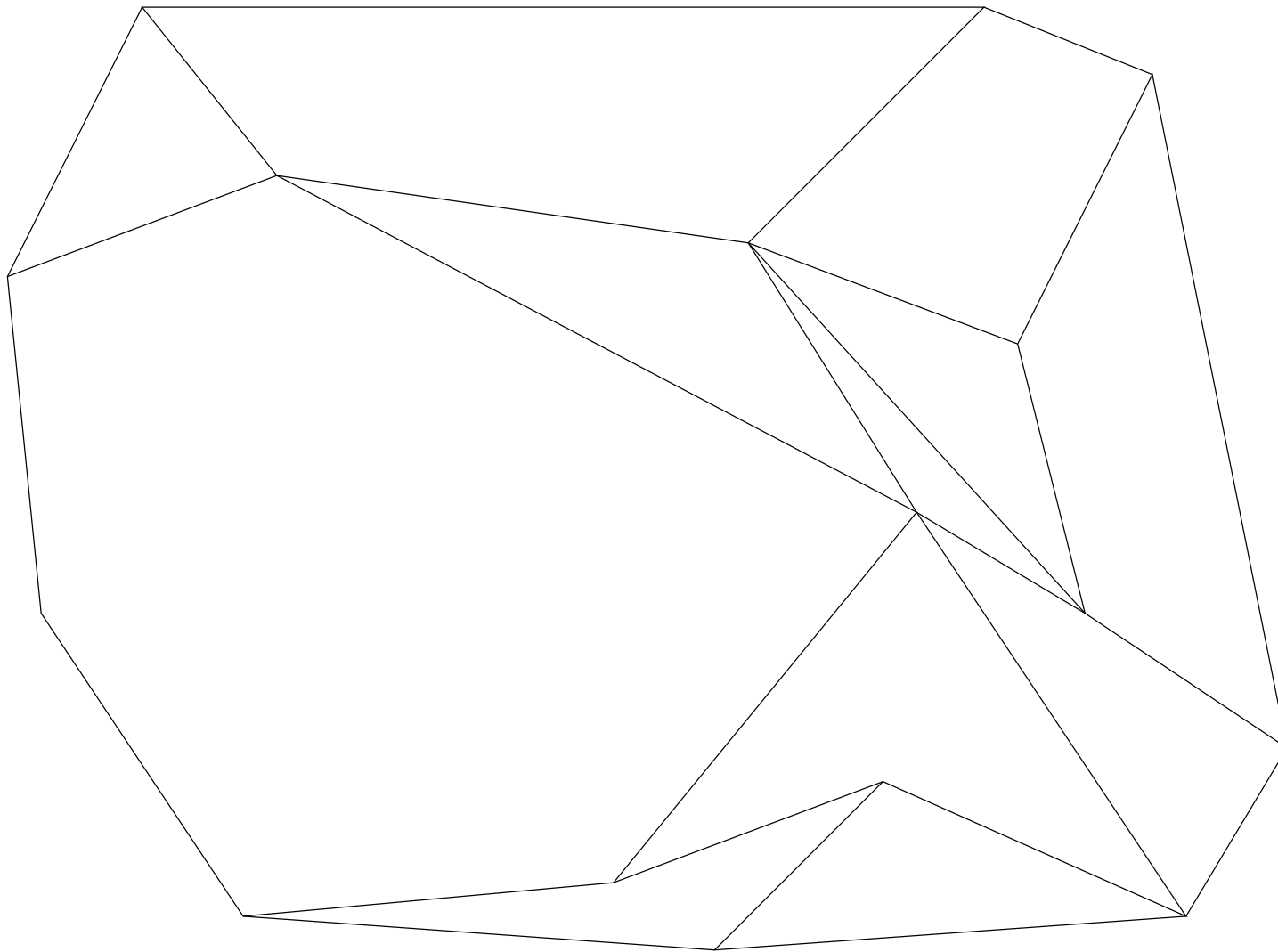
Eingabe: ein eingebetteter planarer Graph $G = (V, E)$

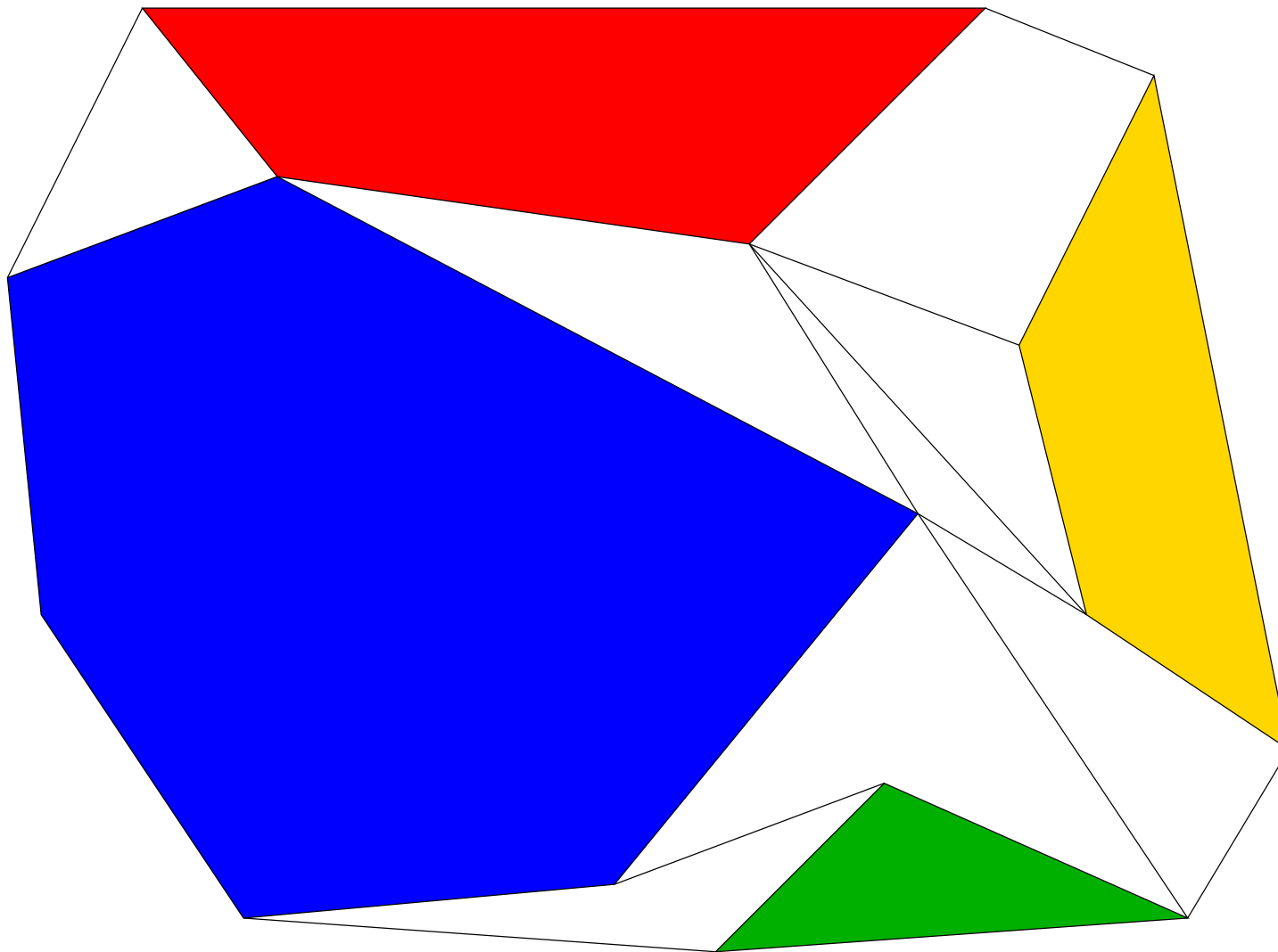
Parameter: eine natürliche Zahl k

Frage: Gibt es eine Facettenüberdeckung der Größe höchstens k ?

Hierbei ist eine **Facettenüberdeckung** eine Menge von Facetten, an deren Grenzlinien alle Knoten des vorgelegten eingebetteten Graphen liegen.

Die folgenden Folien behandeln einen Suchbaumalgorithmus für FC.





Idee & Problem

Verzweigen nach „kleingradigen“ Knoten scheint zunächst möglich nach der Euler-Formel.

ABER: Wir erhalten schon nach dem ersten Verzweigen zwei Arten von Knoten: bereits abgedeckte (marked) und noch abzudeckende (active). Facetten, auf denen wir bereits verzweigt haben, markieren wir ebenso. Wir müssen aber doch nicht auf kleingradigen markierten Knoten verzweigen ?!

$\leadsto$ **Annotierte Facettenüberdeckung**, formalisiert durch Funktionen $\mu_V : V \rightarrow \{active, marked\}$ und $\mu_F : F \rightarrow \{active, marked\}$.

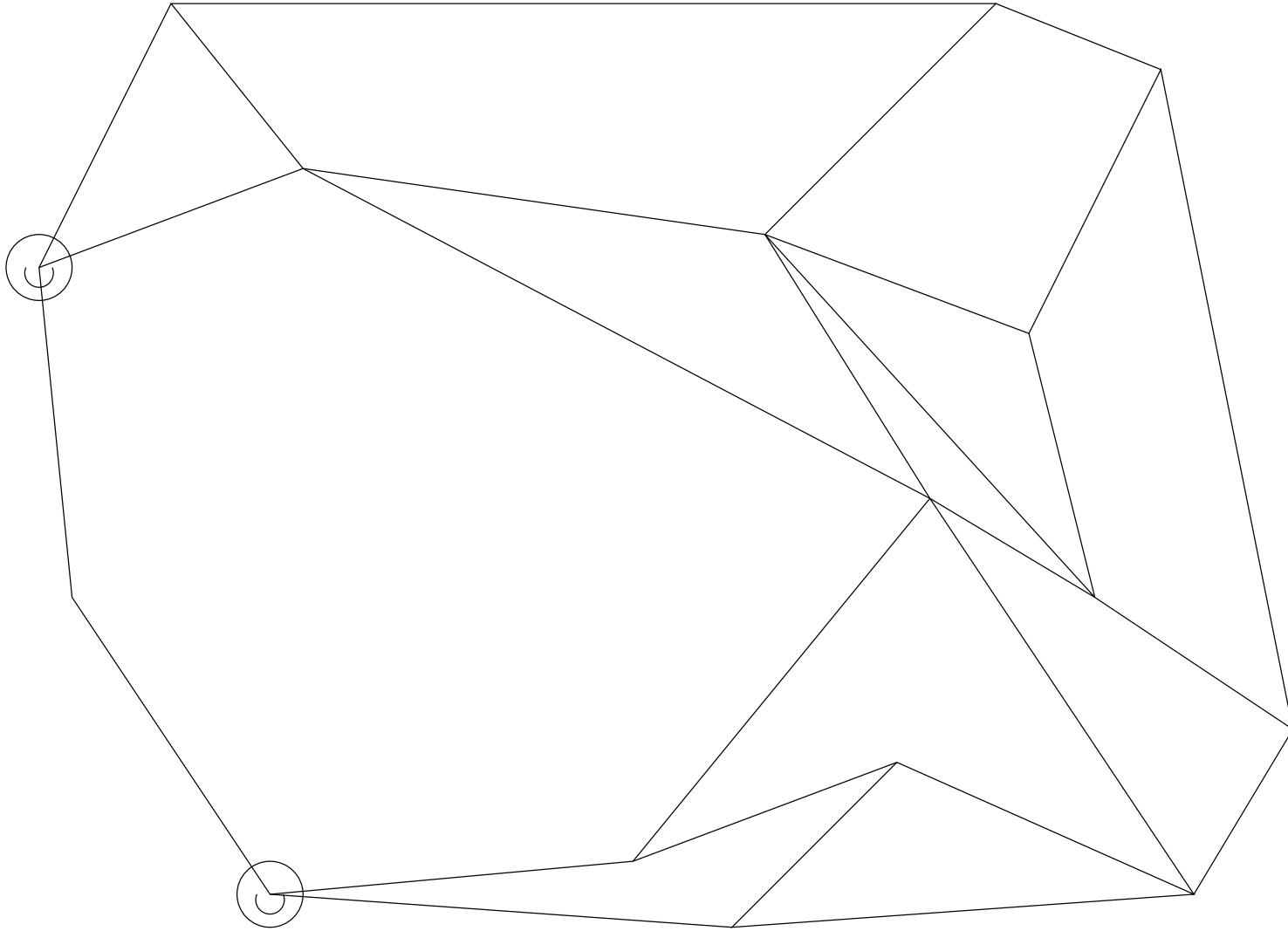
Eingangs sind alle Knoten und Facetten aktiv.

HS-Regeln als Erbschaft

$F_a(v)$: die zu v benachbarten aktiven Facetten

$V_a(f)$: die an f anliegenden aktiven Knoten

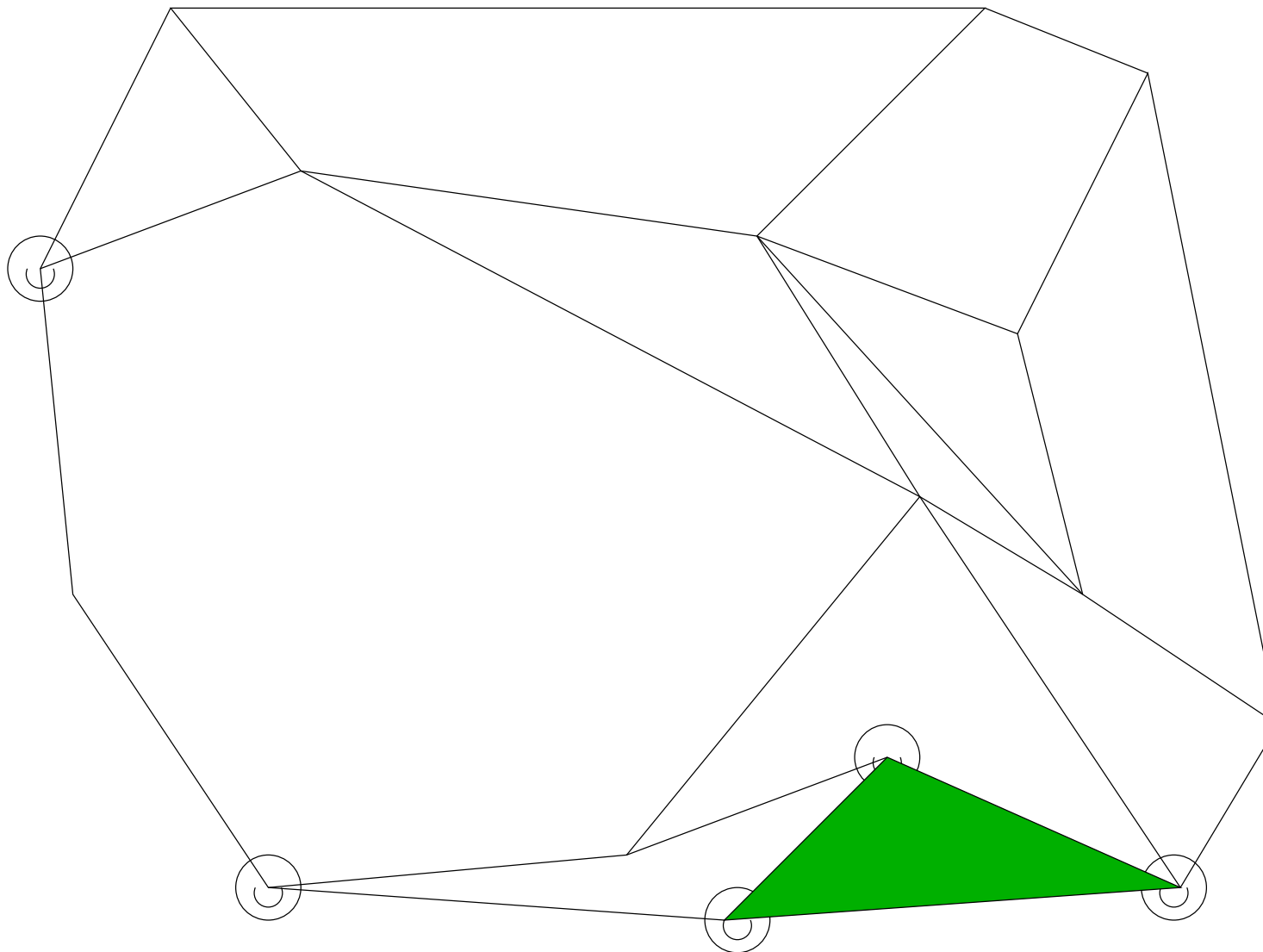
- Gilt $\emptyset \neq F_a(u) \subseteq F_a(v)$ für aktive Knoten u, v , so markiere v .
- Gilt $|F_a(v)| = 1$ und ist v aktiv, so muss $F_a(v)$ in die Facettenüberdeckung; anschließend werden v und f markiert. (Die vorherige Regel war nicht mehr anwendbar.)
- Gilt $V_a(f) \subseteq V_a(f')$ für aktive Facette f, f' , so markiere f .

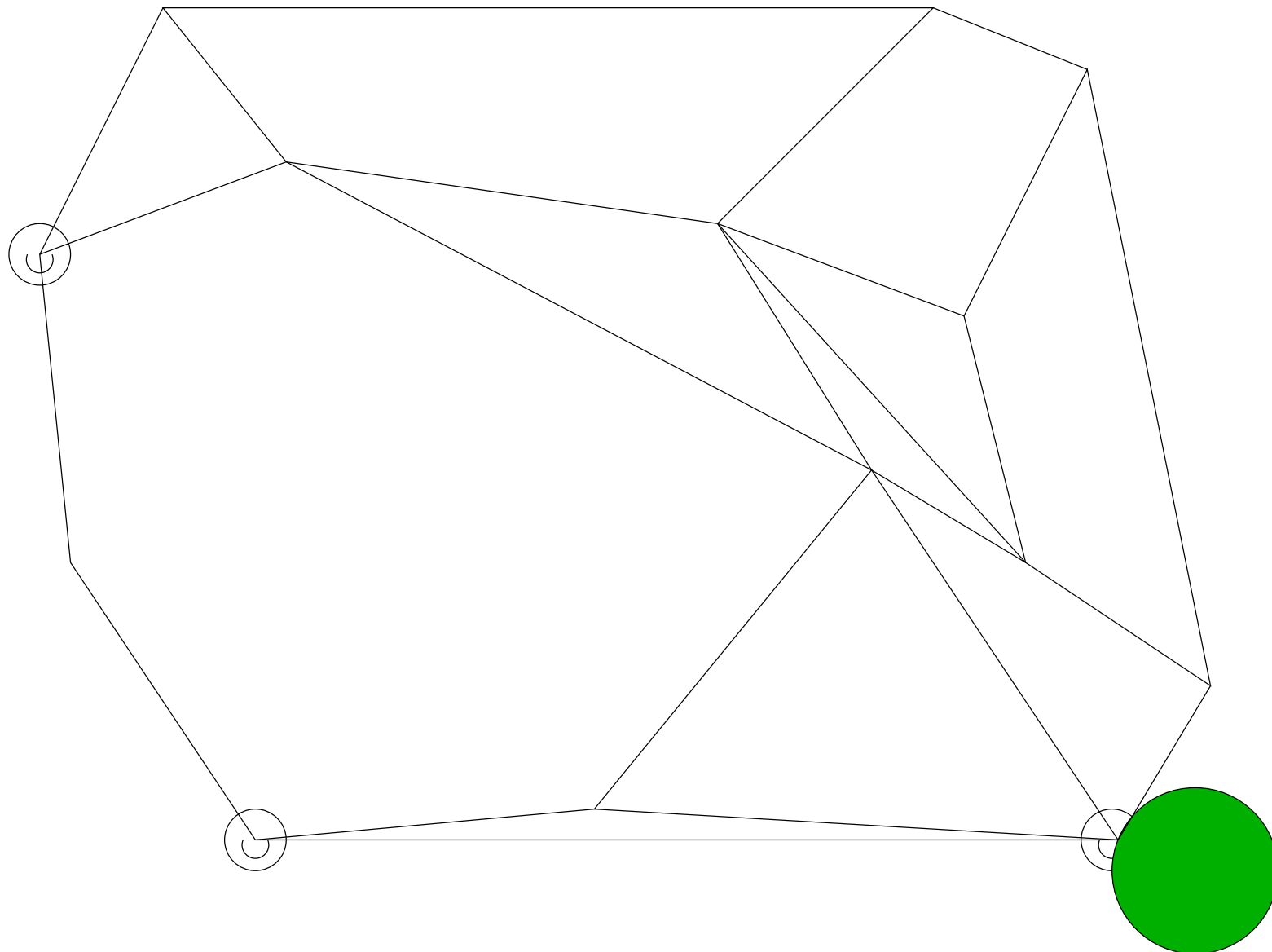


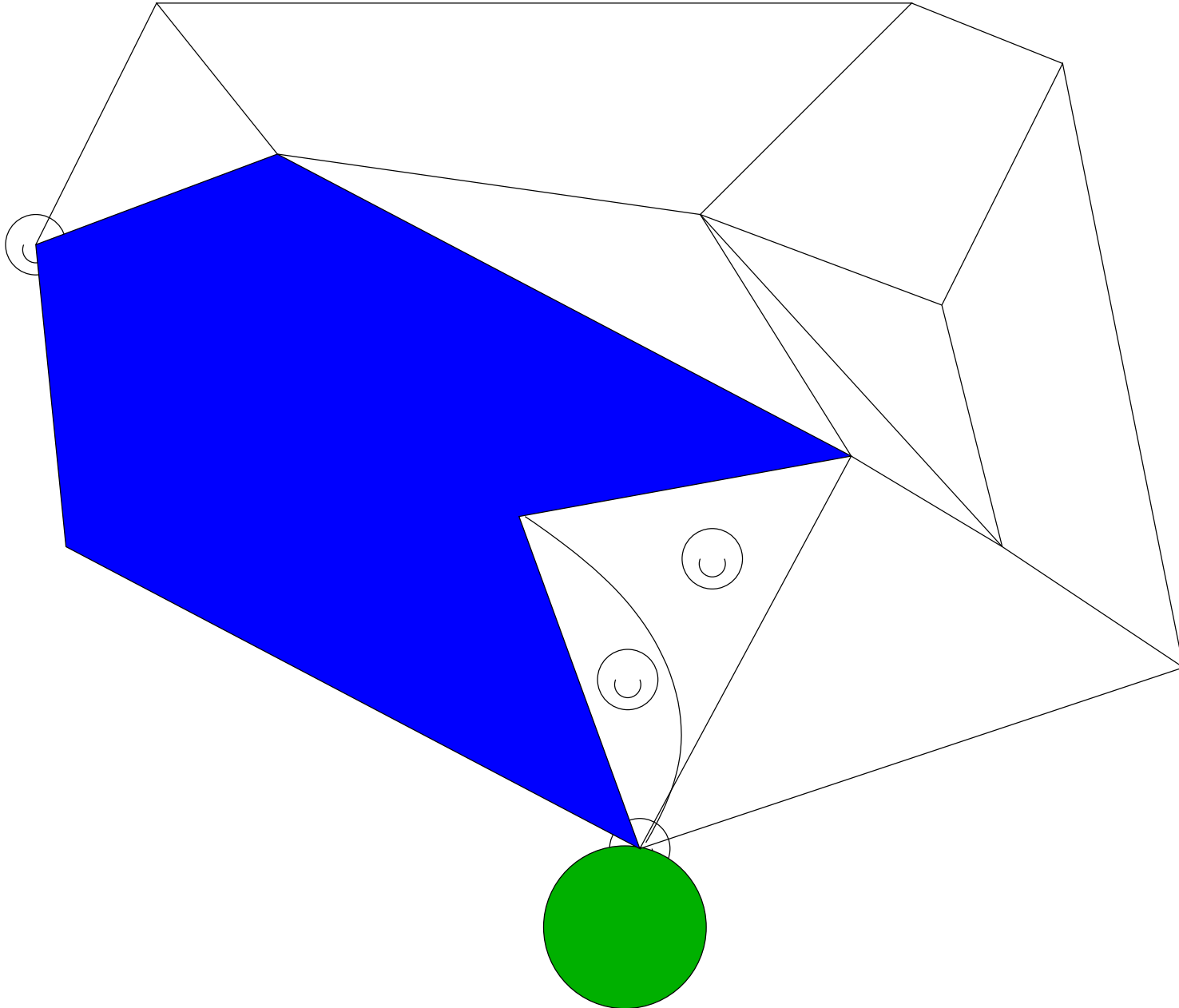
Geometrische Operationen

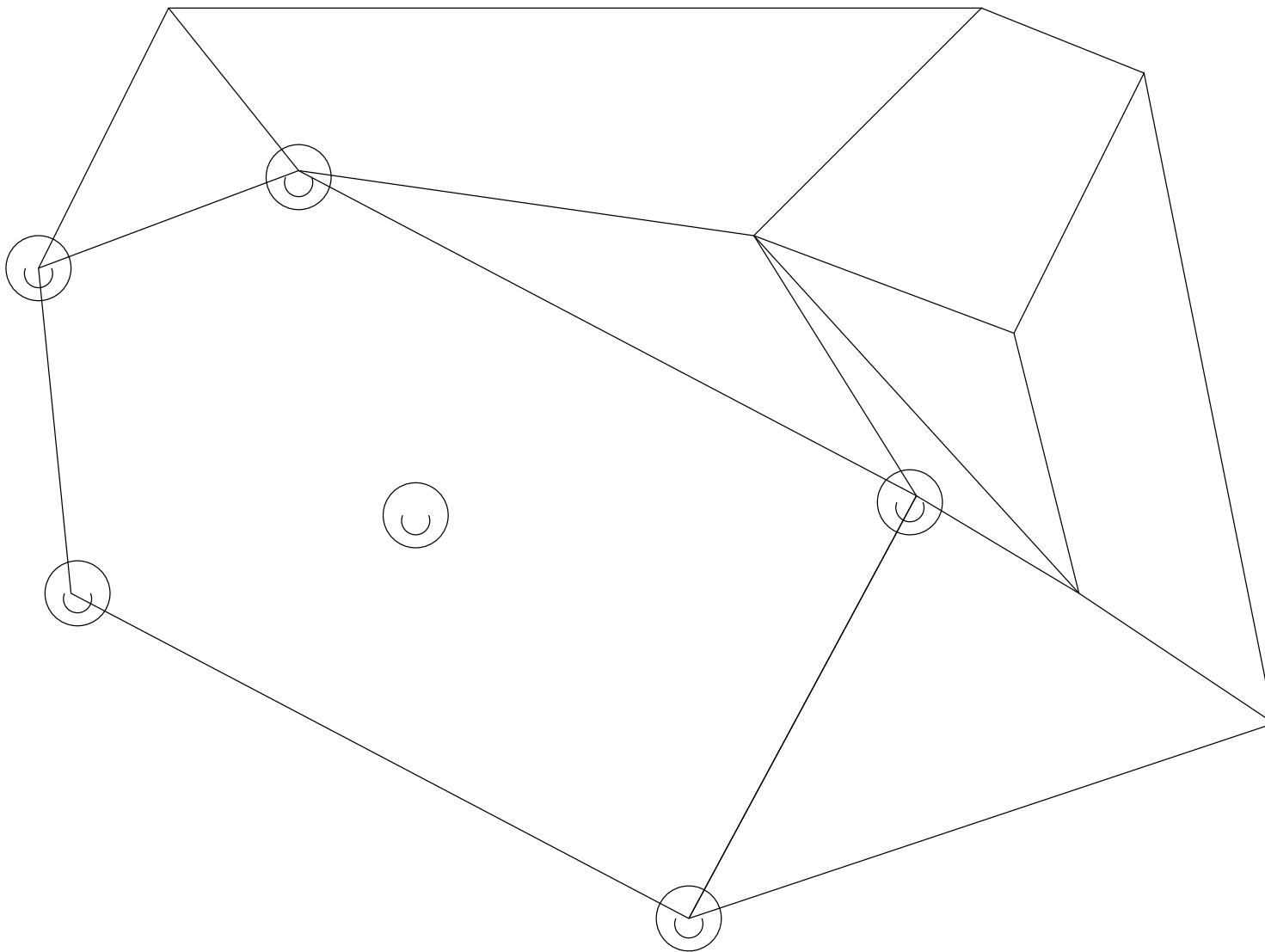
- Sind u und v zwei benachbarte markierte Knoten, so verschmelze u und v (wobei möglicherweise Mehrfachkanten oder Schlingen eingeführt werden).
- Ist u ein markierter Knoten mit zwei aktiven Nachbarn v, w , sodass alle drei Knoten an einer aktiven Facette f anliegen, so unterteile f in zwei Facetten durch Einführung einer neuen Kante zwischen v und w , die innerhalb von f gezeichnet werden muss (wieder kann dies Mehrfachkanten liefern). Das neue Dreieck u, v, w ist markiert, während „der Rest“ von f aktiv ist.
- Lösche markierte Knoten vom Grad Eins.
- Markierte Knoten v mit $|F_a(v)| = 0$ können ebenfalls gelöscht werden. So werden die v zuvor umschließenden markierten Facetten durch eine einzige markierte Facette ersetzt.
- Eine markierte Facette mit nur einem oder zwei Knoten auf ihrer Grenzlinie wird durch Löschen einer beliebig gewählten Begrenzungskante zerstört.
- Zwei markierte Facetten mit einer Grenzkante werden durch Löschen derselben zu einer neuen markierten Facette (u.U.) verschmolzen.

Aufgabe: Begründen Sie die Korrektheit der aufgeführten Regeln!









Algorithm 3 FC-ST: A simple search tree algorithm for annotated FC

Input(s): an annotated plane graph $G = (V, E)$ with face set F and marking functions μ_V and μ_F , a positive integer k

Output(s): YES iff G has an annotated face cover set $C \subseteq F$ with $|C| \leq k$

Exhaustively apply the reduction rules. $\{\rightsquigarrow G$ (etc.) as before.

if $k < 0$ **then**

 return NO

else if $V_a = \emptyset$ **then**

5: return YES

else

 Let v be a vertex of lowest face degree in G .

 Choose $f \in F_a$ such that f is incident to v .

 Mark f and all vertices that are on the boundary of f .

10: Call the resulting marking functions μ'_V and μ'_F .

if FC-ST($G, F, \mu'_V, \mu'_F, k - 1$) **then**

 return YES

else

 Mark f , i.e., return FC-ST(G, F, μ_V, μ'_F, k)

Satz 6 *Algorithmus 3 löst das AFC Problem in Zeit $\mathcal{O}^*(5^k)$.*

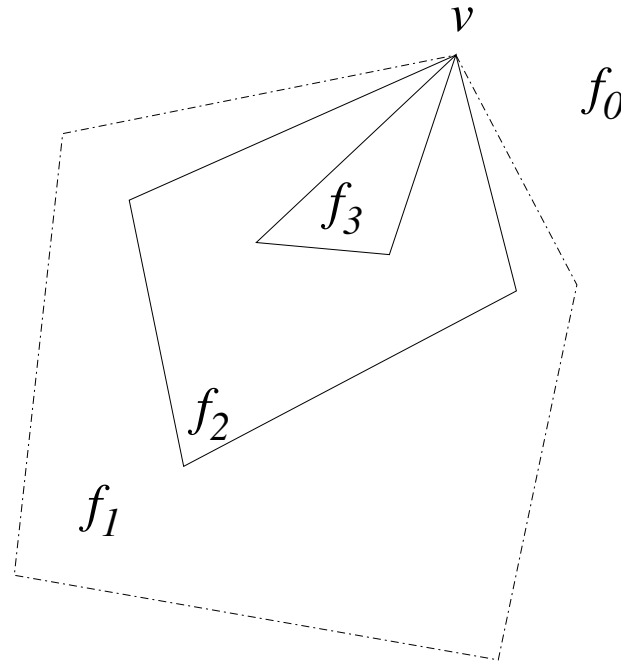
Zu zeigen wäre: In einem annotierten reduzierten (Multi-)graphen G gibt es stets einen Knoten, der an höchstens fünf aktiven Facetten anliegt.

Der Satz von Aksionov et al. gestattet wiederum, die Laufzeit noch deutlich zu verbessern.

Satz 7 *FC kann in Zeit $\mathcal{O}^*(4.6056^k)$ gelöst werden.*

Aufgabe: Machen Sie sich die entsprechende Laufzeitanalyse wenigstens plausibel! Dazu müssen Sie den nachfolgenden Pseudo-Code genauer anschauen.

Was sind Inseln ?



Im Beispiel: Am markierten Knoten v sind drei Schlingen (Multigraph!), die die Ebene in vier Bereiche (Inseln) aufteilen, die unabhängig voneinander gelöst werden können, da v markiert.

Weitere Aussagen

Lemma 8 *Ist $G = (V, E, V_a, F_a)$ ein annotierter, eingebetteter, reduzierter Multigraph ohne Inseln mit Facettenmenge F (als Instanz eines ANNOTATED FACE COVER Problems mit Parameter k), so gilt:*

1. *Es gibt keinen markierten Knoten in G .*
2. *Die einzigen degenerierten Facetten die möglicherweise existieren sind aktive Facetten f mit zwei inzidenten Knoten.*
3. *Jedes Knotenpaar $\{u, v\}$ hat höchstens eine inzidente degenerierte Facette.*
4. *Die beiden zu einer degenerierten Facette über Kanten angrenzenden Facetten sind markiert.*

Zum Beweis des „einfachen“ Satzes:

Zu zeigen: Ist $G = (V, E, V_a, F_a)$ ein annotierter, eingebetteter, reduzierter Multigraph ohne Inseln mit Facettenmenge F (als Instanz eines ANNOTATED FACE COVER Problems mit Parameter k) gibt es stets einen Knoten mit höchstens fünf inzidenten aktiven Facetten.

Betrachte den schlichten eingebetteten Graphen $G' = (V, E')$, der aus G entsteht, indem wir (nur) eine Kante zwischen u und v zeichnen, sobald es irgendeine Kante zwischen u und v in G gibt.

Euler $\leadsto$ In G' können wir einen Knoten vom Maximalgrad fünf finden.

Gemäß dem Lemma könnten der Kante uv in G zwei Kanten (aber nicht mehr) zwischen u und v entsprechen. Somit gäbe es höchstens 10 an v angrenzende Facetten.

Von denen können aber (mit dem Lemma) höchstens 5 aktiv sein.

Algorithm 4 An advanced search tree algorithm for ANNOTATED FACE COVER, called FC-ST-advanced

Input(s): an annotated plane graph $G = (V, E)$ with face set F and marking functions μ_V and μ_F , a positive integer k

Output(s): C if G has an annotated face cover set $C \subseteq F$ with $|C| \leq k$; NO otherwise

Exhaustively apply the reduction rules.

{The resulting instance will be also called G (etc.) as before.}

if $k < 0$ **then**

 return NO

5: **else if** $V_a = \emptyset$ **then**

 return $\emptyset$

else if G contains a marked vertex v whose only neighbor is v itself **then**

 Recursively compute minimum face covers $C_0, \dots, C_r$ of the island graphs $G_0, \dots, G_r$

if $\exists i : C_i = \text{NO} \vee \sum_{i=0}^r |C_i| > k$ **then**

10: return NO

else

 return $\bigcup_{i=0}^r C_i$

else

 {Details on next slide}

Algorithm 5 The non-island-case

Let u be a vertex of lowest face degree in G .
{One incident face of u must be used to cover u .}
if $\deg_a(u) \leq 4$ **then**
 Choose $f \in F_a$ such that f is incident to u .
5: Mark f and all vertices that are on the boundary of f .
 Call the resulting marking functions μ'_V and μ'_F .
 Let $C \leftarrow \text{FC-ST-advanced}(G, F, \mu'_V, \mu'_F, k - 1)$
 if $C \neq \text{NO}$ **then**
 return $C \cup \{f\}$
10: **else**
 Mark (only) f , i.e., return $\text{FC-ST-advanced}(G, F, \mu_V, \mu'_F, k)$
 else
 {Let $N(u) = \{u_1, u_2, v, w, z\}$ be the neighbors of u and similarly $N(v) = \{v_1, v_2, v_3, u, w, z\}$.}
 if all active faces incident with v are degenerate **then**
15: execute FC-ST-case-1
 else if no active faces incident with v are degenerate **then**
 execute FC-ST-case-2
 else
 {all active faces incident with u are degenerate; only one active face incident with v is degenerate}
20: execute FC-ST-case-3

Algorithm 6 The code of FC-ST-case-1

Let $G' = G \setminus \{u, v\}$ and mark the face f to which (formally) u, v belonged;
modify F, μ_V, μ_F accordingly, yielding F', μ'_V and μ'_F .
Let $C \leftarrow \text{FC-ST-advanced}(G', F', \mu'_V, \mu'_F, k - 1)$
if $C \neq \text{NO}$ **then**
 return $C \cup \{f\}$
5: **else**
 for all unordered vertex pairs $\{x, y\}$ such that $x \in N(u) \setminus \{v\}$ and $y \in N(v) \setminus \{x, u\}$ **do**
 Modify μ'_V so that x and y are the only vertices of $N(u) \cup N(v) \setminus \{u, v\}$ that are marked.
 Let $C \leftarrow \text{FC-ST-advanced}(G', F', \mu'_V, \mu'_F, k - 2)$
 if $C \neq \text{NO}$ **then**
10: return $C \cup \{f\}$
 return NO

Algorithm 7 The code of FC-ST-case-2

Let $G' = G \setminus \{u, v\}$ and mark the face f to which (formally) u, v belonged;
modify F, μ_V, μ_F accordingly, yielding F', μ'_V and μ'_F .

for all vertices $x \in \{w, z\}$ **do**

 Modify μ'_V so that x is the only vertex of $N(u) \cup N(v) \setminus \{u, v\}$ that is marked.

 Let $C \leftarrow \text{FC-ST-advanced}(G', F', \mu'_V, \mu'_F, k - 1)$

5: **if** $C \neq \text{NO}$ **then**

 return $C \cup \{f\}$

for all vertices $x \in \{u_1, u_2\}$ and $y \in \{v_1, v_2, v_3\}$ **do**

 Modify μ'_V so that x and y are the only vertices of $N(u) \cup N(v) \setminus \{u, v\}$ that are marked.

 Let $C \leftarrow \text{FC-ST-advanced}(G', F', \mu'_V, \mu'_F, k - 2)$

10: **if** $C \neq \text{NO}$ **then**

 return $C \cup \{f\}$

return NO

Algorithm 8 The code of FC-ST-case-3

```
for all faces  $f \in F_a(u), g \in F_a(v)$  do
    Mark  $f$  and  $g$  and all vertices in  $V_a(f) \cup V_a(g)$ ; call the modified marking functions  $\mu'_V$  and  $\mu'_F$ .
    Let  $C \leftarrow \text{FC-ST-advanced}(G', F, \mu'_V, \mu'_F, k - |\{f, g\}|)$ 
    if  $C \neq \text{NO}$  then
5:       return  $C \cup \{f, g\}$ 
return NO
```

Hinweis: Face Cover wurde im Rahmen einer Diplomarbeit implementiert.

Zusammenfassung Suchbaumentwurf

Grundidee: Gibt es **beschränkte** Kandidatenmengen, aus denen parameterreduzierend ein Element ausgewählt werden muss ?

Beispiele:

Hyperkante bei Hitting Set

Knoten-Nachbarschaft bei PIS

Dabei (wie so) oft **hilfreich**: mathematische Ergebnisse

Bsp.: Existenz kleingradiger Knoten in planaren Graphen

Noch ein Beispiel: Wie ergibt sich hier ein Suchbaumalgorithmus ?
Eine **Clique** ist ein vollständiger Teilgraph.
Ein **Cluster** ist eine Clique, die eine Zusammenhangskomponente ist.
Bei einem **Clustergraph** bildet jede Zusammenhangskomponente eine Clique.

CLUSTERKNOTENPROBLEM (CLUSTER VERTEX DELETION CVD)

Eingabe: Ein Graph $G = (V, E)$

Parameter: $k \in \mathbb{N}$

Frage: Gibt es eine Knotenmenge $D \subseteq V$, sodass $|D| \leq k$ und $G - D$ ist ein Clustergraph ?

Lemma 9 *Enthält G einen induzierten Weg der Länge zwei, so ist G kein Clustergraph (und umgekehrt).*

Aufgabe: Wie sehen also geeignete Kandidatenmengen aus (zum Verzweigen) ?

Zusammenfassung der Grundtechniken

Kerne (Datenreduktion) kennzeichnen FPT

Praktisch sind sie alleine eher Heuristiken zum Lösen schwerer Probleme.

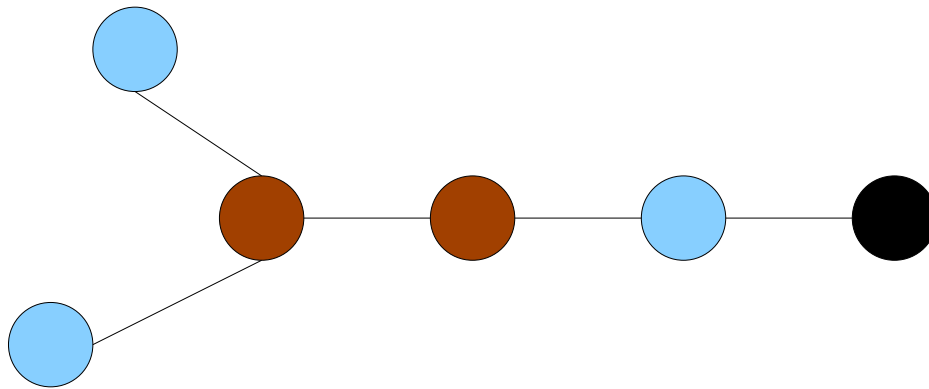
In Kombination mit guten Suchbaumstrategien liefert dies die praktisch besten parameterisierten Algorithmen.

Suchbäume alleine scheinen nicht FPT zu kennzeichnen (**offene Frage !**).

Suchbäume liefern meist den entscheidend(verbessernd)den Anteil für gute Laufzeitabschätzungen.

Parameterisierung nach $n = |V|$:

Berechnung kleinstmöglicher dominierender Mengen



●: in der dominierenden Menge

●: dominiert

●: dominiert; soll nicht in die dominierende Menge

●: noch nicht dominiert; soll nicht in die dominierende Menge

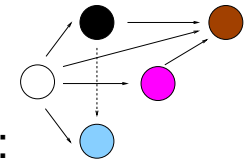
Ist diese Lösung kleinstmöglich?

[illegible]

●: in der dominierenden Menge

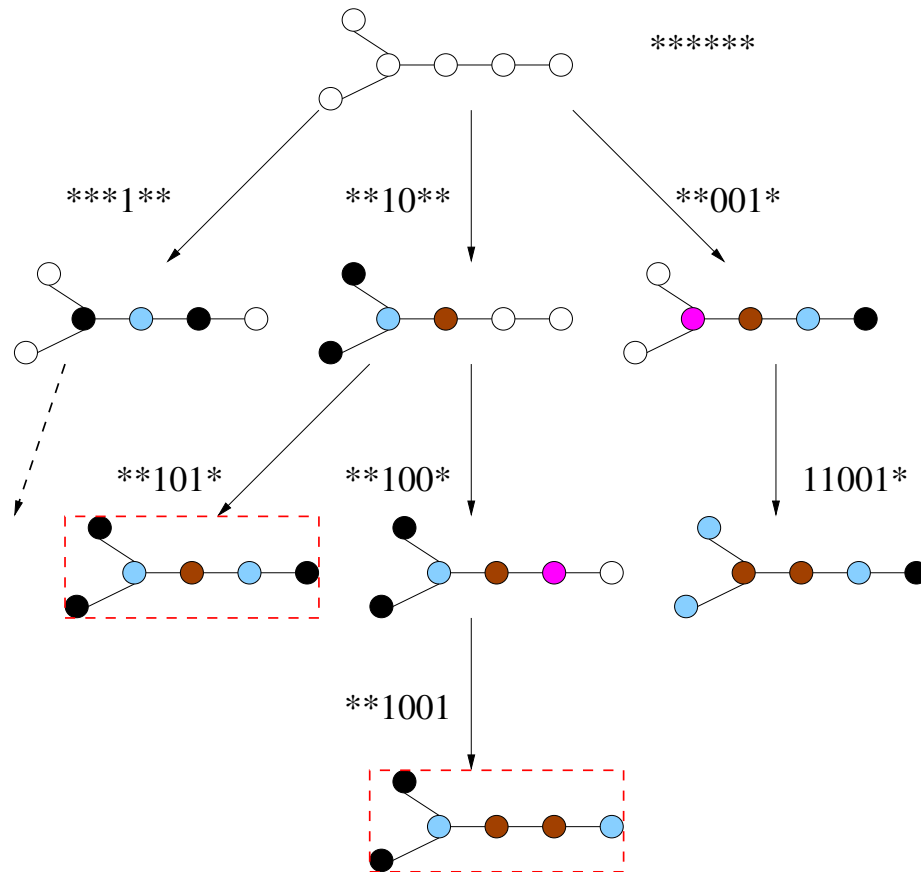
●: dominiert; soll nicht in die dominierende Menge

●: noch nicht dominiert; soll nicht in die dominierende Menge



Verzweige gemäß  oder  Knoten,
solange es  oder  gibt.

Ein farbenfroher Suchbaum



Suchbaum

mit Knotenbeschriftungen.

Beschriftungen kodieren

Verzweigungsentscheidungen.

Je näher zu den Blättern, desto genauer sind die Beschriftungen.

Beobachte: Selbst an den Blättern bestehen die Beschriftungen nicht ausschließlich aus 0 oder 1.

Trivialer Suchbaum?!

Naheliegender Verzweigen auf Knoten:

0: nicht in dominierender Menge

1: kommt in dominierende Menge

Das liefert triviale Abschätzung 2^n der Suchbaumgröße.

Beobachte: Niemals nutzen wir aus, dass wir auch Nachbarn „erobern“, so wir Knoten in die dominierende Menge einfügen.

Offenbar von Vorteil:

Verzweigen auf  im Ggs. zu .

Was ist, wenn es keine , aber noch  und  im Graph gibt?

Gibt es einen Punkt, an dem die Restinstanz in Polynomzeit gelöst werden kann?

Measure & Conquer für DOMINATING SET

Algorithm 9 A simple algorithm for DOMINATING SET

- 1: **if** possible choose a $v \in \bigcirc$ such that $|N(v) \cap (\bigcirc \cup \bullet)| \geq 2$; **then**
 - 2: Binary branch on v (i.e., set v $\bullet$ in one branch, $\bullet$ in the other)
 - 3: **else if** possible choose a $v \in \bullet$ such that $|N(v) \cap (\bigcirc \cup \bullet)| \geq 3$; **then**
 - 4: Binary branch on v .
 - 5: **else**
 - 6: Solve the remaining instance in polynomial time using an EDGE COVER algorithm.
-

Measure & Conquer: DOMINATING SET mit Hilfe von EDGE COVER.

In Schritt 6 erzeuge EDGE COVER Instanz $G_{EC} = (V(E_{EC}), E_{EC})$:

(1a) Für $v \in \bigcirc$ mit $N(v) \cap (\bigcirc \cup \bullet) = \{q\}$,

füge $e = \{v, q\}$ zu E_{EC} und setze $\alpha(e) = v$,

(1b) für $v \in \bullet$ mit $N(v) \cap (\bigcirc \cup \bullet) = \{x, y\}, x \neq y$,

füge $e = \{x, y\}$ zu E_{EC} und setze $\alpha(e) = v$.

Ist C eine kleinstmögliche Kantenüberdeckung von G_{EC} , füge v zu DS, wobei $v = \alpha(e)$ für ein $e \in C$.

(2) Füge alle $v \in \bigcirc \setminus V(E_{EC})$ mit $N(v) \cap (\bigcirc \cup \bullet) = \emptyset$ in DS.

(3) Füge kein $v \in \bullet$ mit $|N(v) \cap (\bigcirc \cup \bullet)| = 0$ in DS.

(4) Füge alle $v \in \bullet$ mit $N(v) \cap (\bigcirc \cup \bullet) = \{s\} \not\subseteq V(E_{EC})$ in DS.

DOMINATING SET: **Analyse der Verzweigungen mit dem Maß**

$$\mu = |\bigcirc| + \omega \cdot (|\bullet| + |\bullet|) \leq n$$

Sei $n_{\bigcirc} = |N(v) \cap \bigcirc|$ und $n_{\bullet} = |N(v) \cap \bullet|$.

Wenn v ins DS in **Schritt 2** kommt, reduziere μ zunächst um 1 (v verschwindet).

$\leadsto$ Knoten in $N(v) \cap \bigcirc$ werden dominiert und kommen daher in die Menge $\bullet$.

$\leadsto \mu$ wird um $n_{\bigcirc} \cdot (1 - \omega)$ reduziert.

Ähnlich: Knoten aus $N(v) \cap \bullet$ werden dominiert und verschwinden so aus μ .

$\leadsto \mu$ wird um $n_{\bullet}\omega$ reduziert.

Kommt v nicht in DS, reduziere μ um $(1 - \omega)$: v wandert aus $\bigcirc$ in $\bullet$.

$\leadsto$ Verzweigungsvektor:

$$(1 + n_{\bigcirc}(1 - \omega) + n_{\bullet}\omega, (1 - \omega)) \quad (1)$$

mit $n_{\bigcirc} + n_{\bullet} \geq 2$ in Schritt 1.

Ähnliche Analyse für **Schritt 4** liefert wegen $n_{\bigcirc} + n_{\bullet} \geq 3$ in Schritt 3:

$$(\omega + n_{\bigcirc}(1 - \omega) + n_{\bullet}\omega, \omega) \quad (2)$$

DOMINATING SET: **Analyse der Verzweigungen in den Schritten 2 und 4.**

Abhängig von $n_{\circ}$ und $n_{\bullet}$ erhalten wir eine *unendliche Zahl von Verzweigungsfällen*. $\leadsto$ Beschränkung auf die schlimmsten Fälle.

Für (1) sind diese gegeben durch $n_{\circ} + n_{\bullet} = 2$ und für (2) $n_{\circ} + n_{\bullet} = 3$.

$\leadsto$ **endliche Zahl von Rekurrenzen** $R_1(\omega), \dots, R_7(\omega)$ abhängig von ω .

Wähle ω so, dass die größte Nullstelle der zugehörigen charakteristischen Polynome minimal wird. In unserem Fall: $\omega := 0.5$.

Der schlimmste Verzweigungsvektor ist nun $(2, 0.5)$.

$\leadsto$ Die Blattanzahl im Suchbaum ist beschränkt durch $\mathcal{O}^*(1.9052^\mu)$.

$\leadsto$ Der Algorithmus bricht die 2^n -Schranke mit einem einfachen Maß.

Aufgabe: Konstruieren Sie ein Beispiel, bei dem nach dem Verzweigen eine nicht-triviale EDGE COVER-Instanz im Sinne von „Triviality Last“ übrigbleibt.

Aufgaben zur Übung

Wie können M&C-Analysen aussehen für das Problem, eine kleinstmögliche dominierende Menge $D \subseteq V$ in einem Graphen $G = (V, E)$ zu bestimmen, so dass zusätzlich

- D eine unabhängige Menge ist, oder
- $G[D]$ ist zusammenhängend oder
- $G[V \setminus D]$ ist zusammenhängend ?

Finden Sie jeweils Algorithmen mit Analysen, die Laufzeiten $O(c^{|V|})$ zeigen für $c < 2$. Warum wurde der Fall „ $V \setminus D$ sei unabhängig“ nicht gesondert betrachtet?